

TriMouNet User Manual

Mao Qiyun

Overview

This manual describes a complete TriMouNet workflow in three stages:

1. Quartet extraction and preprocessing (`extract_outgroup_quartets.py`): extract all outgroup-rooted quartets from full gene trees, clean/normalize branch lengths, filter outliers, and generate a branch-length-free, numeric-encoded file for fast split counting.
2. Parameter estimation and TNET generation (`infer_trinet_types_and_generate_tnets.py`): estimate model parameters from quartets (or quartet-derived statistics) and aggregate them into a *.tnets file compatible with TriMouNet.
3. Network reconstruction (`TriMouNet.jar`): infer the final phylogenetic network from the *.tnets input and output the network topology and associated parameters (e.g., `w1` for reticulated cherries).

Part I — Quartet extraction and preprocessing (`extract_outgroup_quartets.py`)

I.1 Purpose

The goal of this script is to convert full gene trees (spanning all taxa) into outgroup-rooted quartet trees required by downstream TriMouNet processing. In addition, it applies a sequence of cleaning and normalization steps to ensure:

Quartet trees are comparable (consistent Newick layout with outgroup placement fixed);

Branch-length values are numerically stable (avoid exact zeros and pathological rooting artifacts);

An auxiliary file is produced with branch lengths removed and taxa mapped to integers, enabling efficient

split counting in later stages.

I.2 Inputs

a text file containing Newick gene trees, one tree per line, each ending with ; The script assumes each input tree contains the complete terminal set {A..H, O} (the ingroup taxa + outgroup O, i.e., 9 terminals).

In practice, users should first reconstruct gene trees for multiple loci (e.g., using IQ-TREE on per-locus alignments). The resulting locus-wise maximum-likelihood trees should then be concatenated into a single .tre file, where each line corresponds to one locus tree in Newick format. This concatenated multi-locus tree file is used as the input to the quartet-extraction script.

I.3 Outputs

The script produces intermediate files for transparency and debugging:

- ***_quartet.tre** (Step 1): all pruned quartet trees containing {triplet + O}, written with fixed branch-length precision.
- ***_quartet_adjust.tre** (Step 2): quartets after branch-length cleanup and root–outgroup adjustment.
- ***_deleteoutlier.tre** (Step 3): quartets after removing outlier branch-length configurations.
- ***_modify.tre** (Step 4): normalized Newick with outgroup O placed consistently on the outside.
- ***_withoutweight_num.tre** (Step 5): branch-length–stripped, outgroup-removed, and numeric-encoded taxa (A..H → 1..8), for downstream split statistics.

I.4 Step-by-step description

Step 1 — Quartet extraction

Enumerate all 3-taxon subsets of the ingroup and prune each full gene tree down to {triplet + O}.

Step 2 — Branch-length cleanup and root–outgroup adjustment

- Replace exact zero-length branches with 0.00001 to avoid invalid or collapsed trees.

- If the root has exactly two child clades and one child is the outgroup O, then push the internal root edge length onto the outgroup edge and set the subtree root length to 0. This prevents pruning-induced artifacts where O becomes artificially close to the root.

Step 3 — Outlier filtering

Remove quartets with anomalous branch-length patterns (e.g., extremely short branches or a single overly dominant branch), which are likely numerical artifacts and can distort downstream parameter estimation.

Step 4 — Newick normalization (fixing outgroup placement)

Rewrite Newick strings so that outgroup O appears in a consistent, standardized position (e.g., as the outermost first child).

Step 5 — Remove branch lengths and encode taxa as integers

Strip all branch lengths, remove O, and map A..H to 1..8. This produces a compact file optimized for split counting.

Part II—Parameter estimation and TNET generation (infer_trinet_types_and_generate_tnets.py)

II.1 Purpose

This stage converts quartet trees (or quartet-derived statistics) into a **dense** .tnets input file for TriMouNet. It consists of:

1. estimating model parameters for each local structure (trinet/quartet-derived unit);
2. aggregating multi-locus evidence into final parameters using the TriMouNet merging/weighting rules;
3. writing a .tnets file containing one entry per 3-taxon subset (dense input), plus a loci-count line (Num).

II.2 Inputs

Depending on your implementation, typical inputs include:

- the normalized quartet file from Part I (*_modify.tre)
- the numeric-encoded file (*_withoutweight_num.tre) for split counting,
-

II.3 Outputs: .tnets file format

A .tnets file is a whitespace-delimited text file. Each trinet line begins with three taxa labels and a trinet type, followed by the parameters required by that type.

A B C T1 <param1> <param2> ... p_value_second p_value_first

A D B S1 <param1> <param2> ... p_value_second p_value_first

and the final file ends with a line recording the locus count, e.g.:

N U M Num 4000 (The number of loci)

II.4 Step-by-step description

Step 1 — Enumerate all ingroup triplets (8 choose 3)

Step 2 — Count split/topology frequencies from *_withoutweight_num.tre

Step 3 — Parameter fitting by optimization (BFGS / L-BFGS-B)

Let X_1 and X_2 be independent random variables following exponentially modified Gaussian distribution (EMG, also known as exGaussian distribution). $X_1 \sim \text{EMG}(\mu_1, \sigma_1^2, \lambda_1)$ and $X_2 \sim \text{EMG}(\mu_2, \sigma_2^2, \lambda_2)$.

Therefore, a mixture $wX_1 + (1-w)X_2$ has seven free parameters, which are difficult to optimize in practice. We reduce the dimension of the problem to three by assuming $\lambda := \lambda_1 = \lambda_2$, $\sigma := \sigma_1^2 = \sigma_2^2$, and by assuming that the mean and the variance of the mixture equals the sample mean and the sample variance, respectively. In addition, we

transform the remaining variables to simplify the search spaces.

Let X denote a random variable drawn from a two-component mixture of X_1 and X_2 . Let S_m and S_v denote the mean and variance of X , respectively, while $S_{m1}, S_{m2}; S_{v1}, S_{v2}$ and w_1, w_2 denote the means, the variances and the weights of the first component and the second component;

In the exGaussian model, μ denotes the mean of Gaussian component, σ^2 denotes the variance of Gaussian component, λ denotes the rate parameter of the exponential component. We reparameterize the model by defining $x=1/\lambda$, which is used as the optimized parameter. Since the expectation of an EMG equals $\mu+1/\lambda$, we have

$$E(X)=S_m= w_1S_{m1}+ w_2S_{m2}=w_1(\mu_1+x)+w_2(\mu_2+x)=w_1\mu_1+w_2\mu_2+x.$$

Since the variance of an EMG equals σ^2+1/λ^2 , the variance $D(X)$ of X equals

$$\begin{aligned} D(X) &= S_v = E(X^2) - S_m^2 \\ &= w_1(\sigma_1^2 + \mu_1^2) + w_2(\sigma_2^2 + \mu_2^2) + 2S_mx - S_m^2 \end{aligned}$$

This implies

$$S_v + S_m^2 = w_1(\sigma_1^2 + \mu_1^2) + w_2(\sigma_2^2 + \mu_2^2) + 2S_mx \quad (1)$$

To enforce the linear constraint $w_1\mu_1+w_2\mu_2=C=S_m \cdot x$, we introduce a one-dimensional parameterization

$$\mu_1=C-w_2t, \mu_2=C+w_1t \quad (2), \text{ which satisfies the constraint for all } t \in \mathbb{R}.$$

Substituting (2) into (1), we obtain

$$w_1\sigma_1^2 + w_2\sigma_2^2 + C^2 + w_1w_2t^2 = S_v + S_m^2 - 2S_mx \quad (3)$$

From (3), we have $S_v + S_m^2 - 2S_mx \geq (S_m - x)^2 = C^2$, thus

$$x \leq \sqrt{S_v}$$

Moreover, the non-negativity of the left-hand side of (3) yields

$$x \leq \frac{S_v}{2S_m} + \frac{S_m}{2}$$

Combining this result with the previous bound, we obtain

$$x \leq \min\left(\sqrt{S_v}, \frac{S_v}{2S_m} + \frac{S_m}{2}\right)$$

From (1) and (3), we get

$$w_1\mu_1^2 + w_2\mu_2^2 = C^2 + w_1w_2t^2 \quad (4)$$

$$\text{Let } y^* = \frac{w_1\sigma_1^2 + w_2\sigma_2^2}{S_v + S_m^2 - 2S_mx}, \quad t^2 = \frac{(S_v + S_m^2 - 2S_mx)(1 - y^*) - C^2}{w_1w_2} \quad (5)$$

Since $w_1 + w_2 = 1$, the AM–GM inequality implies that $w_1w_2 \leq 1/4$

Consequently,

$$t \geq \sqrt{(S_v + S_m^2 - 2S_mx)(1 - y^*) - C^2} \quad (6)$$

From (5), we get

$$(S_v + S_m^2 - 2S_mx)(1 - y^*) - C^2 \geq 0 \Rightarrow y^* \leq \frac{S_v - x^2}{(S_v + S_m^2 - 2S_mx)}$$

Let $y = y^* \frac{S_v + S_m^2 - 2S_mx}{S_v - x^2}$, so $y \leq 1$

From (2) and (5), we could get

$$\mu_1 = S_m - x - \sqrt{\frac{((S_v + S_m^2 - 2S_mx)(1 - y^*) - C^2)w_2}{w_1}} \quad (7)$$

From (7) we could get

$$S_m - x \geq \sqrt{\frac{((S_v + S_m^2 - 2S_mx)(1 - y^*) - C^2)w_2}{w_1}},$$

$$\frac{((S_v + S_m^2 - 2S_mx)(1 - y^*))w_2}{(S_m - x)^2} \leq 1$$

$$\text{Let } w = \frac{((S_v + S_m^2 - 2S_mx)(1 - y^*))w_2}{(S_m - x)^2}, \quad 0 < w < 1 \Rightarrow$$

$$\begin{aligned} w_1 &= \frac{w(S_m - x)^2}{(S_v + S_m^2 - 2S_mx)(1 - y^*)} \\ &= 1 - \frac{w(S_m - x)^2}{S_v + S_m^2 - 2S_mx - S_v y + x^2 y} \end{aligned}$$

$$\begin{aligned}
w_2 &= \frac{w(S_m - x)^2}{((S_v + S_m^2 - 2S_mx)(1 - y^*))} \\
&= \frac{w(S_m - x)^2}{S_v + S_m^2 - 2S_mx - S_vy + x^2y}
\end{aligned} \tag{8}$$

From (5), we get

$$\begin{aligned}
\sigma_1 = \sigma_2 &= \sqrt{y^* (S_v + S_m^2 - 2S_mx)} \\
&= \sqrt{y(S_v - x^2)}
\end{aligned}$$

From (5) and (8), we express $\mu_1 = C - w_2t$, $\mu_2 = C + w_1t$ by using x , y , and w .

After this reparameterization, we obtain three independent variables x , y , and w which can be optimized without constraints among them. We apply the L-BFGS-B algorithm in this parameter space. After this transformation, the optimization procedure achieves stable convergence and reduces the risk of convergence to local optima or spurious solutions.

Step 4 — Write the final.tnets file (trinet type assignment)

Part III—Network reconstruction (TriMouNet.jar)

II.1 Purpose

This step uses TriMouNet to convert the inferred trinet set (.tnets) into a single phylogenetic network over all taxa, exported as a DOT graph for visualization with GraphViz.

II.2 Command

Run TriMouNet

From the command line, run TriMouNet and specify an output name/prefix:

```
java -jar TriMouNet.jar -i your_dataset.tnets -o myOutput.dot
```

Visualize with GraphViz

Render the DOT file to a publication-ready figure using GraphViz:

```
dot -Tpdf myOutput.dot -o myOutput.pdf
```

or

```
dot -Tpng myOutput.dot -o myOutput.png
```